

Fractional Knapsack

Last Updated : 02 Sep, 2025



Given two arrays, `val[]` and `wt[]`, representing the values and weights of item respectively, and an integer `capacity` representing the maximum weight a knapsack can hold, we have to determine the **maximum total value** that can be achieved by putting the items in the knapsack without exceeding its capacity.

Items can also be taken in fractional parts if required.

Examples:

Input: `val[] = [60, 100, 120]`, `wt[] = [10, 20, 30]`, `capacity = 50`

Output: 240

Explanation: We will take the items of weight 10kg and 20kg and $\frac{2}{3}$ fraction of 30kg.

Hence total value will be $60 + 100 + (\frac{2}{3}) * 120 = 240$.

Input: `val[] = [500]`, `wt[] = [30]`, `capacity = 10`

Output: 166.667

Why naive approach will fail?

Case 1: Picking the items with smaller weights first

Example: `val[] = [10, 10, 10, 100]`, `wt[] = [10, 10, 10, 30]`, `capacity = 30`

- If we start picking smaller weights, we can pick the three items of weight 10 → total value = $10+10+10 = 30$.
- But the optimal choice is to take the last item (value 100, weight 30) → total value = 100.

So, choosing smaller weights first fails here.

Case 2: Picking items with larger value first

Example: `val[] = [10, 10, 10, 20]`, `wt[] = [10, 10, 10, 30]`, `capacity = 30`

- If we start picking higher values, we might choose the last item (value 20, weight 30) → total value = 20.

- But the better choice is to take the three items of weight 10 each → total value = $10+10+10 = 30$.

So, choosing higher values first also fails.

[Approach] Selecting Items by value/weight Ratio - $O(n \log n)$ Time and $O(n)$ Space

*The idea is to always **pick items greedily** based on their **value-to-weight ratio**. Take the item with the highest ratio first, then the next highest, and so on, until the knapsack is full. If any item doesn't fully fit, then take its fractional part according to the remaining capacity.*

Steps to solve the problem:

1. Calculate the ratio (value/weight) for each item.
2. Sort all the items in decreasing order of the ratio.
3. Iterate through items:
if the current item fully fits, add its full value and decrease capacity otherwise, take the fractional part that fits and add proportional value.
4. Stop once the capacity becomes zero.

C++

Java

Python

C#

JavaScript

```
#include <iostream>
#include<vector>
#include<algorithm>
using namespace std;

// Comparison function to sort items based on value/weight ratio
bool compare(vector<int>& a, vector<int>& b) {
    double a1 = (1.0 * a[0]) / a[1];
    double b1 = (1.0 * b[0]) / b[1];
    return a1 > b1;
}

double fractionalKnapsack(vector<int>& val, vector<int>& wt, int capacity) {
    int n = val.size();
```

```

// Create 2D vector to store value and weight
// items[i][0] = value, items[i][1] = weight
vector<vector<int>> items(n, vector<int>(2));

for (int i = 0; i < n; i++) {
    items[i][0] = val[i];
    items[i][1] = wt[i];
}

// Sort items based on value-to-weight ratio in descending
order
sort(items.begin(), items.end(), compare);

double res = 0.0;
int currentCapacity = capacity;

// Process items in sorted order
for (int i = 0; i < n; i++) {

    // If we can take the entire item
    if (items[i][1] <= currentCapacity) {
        res += items[i][0];
        currentCapacity -= items[i][1];
    }

    // Otherwise take a fraction of the item
    else {
        res += (1.0 * items[i][0] / items[i][1]) *
currentCapacity;

        // Knapsack is full
        break;
    }
}

return res;
}

int main() {
    vector<int> val = {60, 100, 120};
    vector<int> wt = {10, 20, 30};

```

```
int capacity = 50;

cout << fractionalKnapsack(val, wt, capacity) << endl;

return 0;
}
```

Output

240

Fractional Knapsack Problem

 Comment

K kartik

 305 

Article Tags :

Greedy

DSA

knapsack

Fraction

Explore

DSA Fundamentals



Data Structures



Algorithms



Advanced



Interview Preparation



Practice Problem



Do Not Sell or Share My Personal Information